7:21 PM 50%

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternat... 4 Mar
To You

Dear Sreetama Santra ,

Project: Smart Course Registration & Academic Planner System
Decision: APPROVED_MINOR
Score: 90

Strengths:
1. **Clear Problem Definition**: The proposal articulates a realistic and common set of problems in college registration systems, providing strong justification for the project and clearly outlining what issues it aims to solve.
2. **Robust OOP Focus**: Explicitly identifies key OOP concepts (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections) and outlines how they will be applied to model

Reply

the domain and enforce rules dynamically, demonstrating a strong grasp of core principles.

3. **Well-Structured Architecture**: Proposes a sound layered architecture (Model, Service, Repository, UI) that aligns with best practices for separation of concerns, modularity, and maintainability, which is excellent for a university project.
4. **Comprehensive Feature Set**: The listed features cover essential functionalities, including advanced validations (credit limits, prerequisites, timetable clashes, capacity management), academic planning, and administrative tasks, making for a complete system simulation.
5. **Emphasis on Dynamic Validation**: The commitment to dynamically validate constraints at the object level (e.g., credit limits, prerequisites) demonstrates a strong understanding of how OOP can be leveraged for robust business logic enforcement.
6. **Appropriate Scope**: The project's scope is challenging enough to thoroughly demonstrate advanced OOP principles and software design patterns without being overly ambitious for a university assignment.

Reply

handling is acceptable for a university project.

Improvements:

1. **Persistence Mechanism**: While file handling is acceptable for a university project, consider elaborating on the specific file handling strategy (e.g., plain text, CSV, Java object serialization) or briefly mentioning the potential benefits of a lightweight embedded database (like H2 or SQLite) for robustness and scalability, even if not fully implemented.
2. **Testing Strategy**: The proposal could benefit from mentioning a plan for unit or integration testing, especially given the complexity of the business logic and exception handling, which would ensure system reliability and correctness.
3. **Polymorphism in Practice**: While polymorphism is listed, providing a brief example of how different `Course` or `User` subclasses might exhibit polymorphic behavior in specific methods (e.g., `calculateFee()` might differ for `UndergraduateCourse` vs. `GraduateCourse`) could further strengthen this section.

Regards,
Course Instructor